Audio Spectrum Analyzer

Luciani Stefano

Academic Year 2025/2026

Abstract

This project presents the design, reproduction, and simulation of an **Audio analyzer** using **AMD Vivado Design Suite**.
The system architecture leverages IP blocks and Block design configurations in order to obtain the most optimized system possible. All the simulation are performed using **ILA (Integrated Logic Analyzer)** and Matlab.

The motivation of this project is applying in practice concepts of telecommunication and electronics in the field of digital signal processing that i have studied during this three years. The design of the system followed online resources and my own telecommunication background in Transmission of Digital Signals and Signals Theory. The objective of the project is extracting Amplitude and Frequency information from an Input analog signal through an omnidirectional MEMS microphone and digital filters.
The microphone uses an Analog Device ADMP421 chip which has a high signal to noise ratio (SNR) of 61dBA and high sensitivity of -26 dBFS.

This report provides a rigorous mathematical background in the field of digital signal processing, a detailed characterization of the steps to tune the system under **system constraints** and a comprehensive analysis of the blocks that are used to build it.

Contents

1 Project information **2**
- 1.1 Theoretical backgrounds in telecommunications . . . 2
- 1.2 Project constraints . . . 2

2 Filter Theory **3**
- 2.1 CIC filter theory . . . 3
- 2.2 FIR filter theory . . . 5

3 FIlter design On MATLAB **5**
- 3.1 CIC filter design . . . 5
- 3.2 FIR filter design . . . 5

4 Data analysis on MatLab **6**
- 4.1 Decimation . . . 6
- 4.2 FIR filter . . . 6
- 4.3 FFT analysis . . . 7

5 Block Design functionalities **8**
- 5.1 Hardware Synchronization: The Processor System Reset Module . . . 8
- 5.2 Functional Necessity and the Metastability Problem . . . 8
- 5.3 Architectural Implementation: Asynchronous Assert, Synchronous De-assert . 8
- 5.4 Impact on the AXI-Stream Data Path . . 8
- 5.5 Analysis of the IP Reset Configuration Parameters . . . 8
- 5.6 Clock Divider . . . 8
- 5.7 PDM wrapper (axi stream adapter) . . . 9
- 5.8 CIC filter . . . 9
- 5.9 FIR filter . . . 9

6 IP Core Parameter Configuration: FIR Compiler **9**
- 6.1 Detailed Implementation Options . . . 9
- 6.2 Architectural Design Justifications . . . 10
- 6.3 FFT IP core . . . 10
- 6.4 FFT IP core: introduction to Arithmetic configuration . . . 10
  - 6.4.1 The Unscaled Arithmetic Mode and Bit Growth Management . . . 10
  - 6.4.2 The Truncation Rounding Mode and Critical Path Optimization . . 11
  - 6.4.3 Lower Bit Selection and Geometric Impact on the Display . . . 11
- 6.5 Configuration and Hardware Architecture of the FFT Compiler IP Core . . . 11
  - 6.5.1 Choice of Hardware Architecture: Pipelined Streaming I/O . . . 12
  - 6.5.2 Arithmetic Configuration: Analysis of Unscaled and Truncation Parameters . . . 12
  - 6.5.3 Optimization of Complex Multipliers Using a Three-Multiplier Structure . . . 12
  - 6.5.4 Boundary Conditions for Real Input and Propagation in the Complex Domain . . . 13
- 6.6 Spectral Energy Computation and Bit-Slicing Architecture . . . 13
- 6.7 Spectrum Isolator and Synchronous Memory Addressing . . . 13
- 6.8 DiSplay the FFT spectrum via VGA signal 14
- 6.9 Frequency Mapping and BRAM Addressing 14

7 Module VGA_Display **15**
- 7.1 General Architecture . . . 15

7.2 Phase 1 — Geometric Screen Mapping . . 15
7.3 Phase 2 — Peak Hold Update 15
7.4 Phase 3 — Color Generation for Each Pixel 15
7.5 Temporal Dynamics and Visual Perception 16

8 POWER CONSUMPTION 16

9 Timing Report 17
9.1 Critical Paths 17

10 ILA analysis 17

11 Conclusion 18

1 Project information

1.1 Theoretical backgrounds in telecommunications

In order to analyze an audio signal, we have to consider three main concepts of telecommunication:

1. **The Nyquist-Shannon Theorem**: to avoid the aliasing phenomenon (spectral overlap), the sampling frequency (fs) must be at least twice the maximum frequency contained in the analog signal (fmax). This means that
$$f_s \geq 2f_{\max}$$
2. **Over Sampling**: Unlike traditional converters, the Nexys 4 DDR digital microphone uses Pulse Density Modulation (PDM). This shifts the quantization noise towards high frequencies (Noise Shaping). The resulting signal is 1 bit, where the information is contained in the density of the pulses rather than in their binary amplitude. For the microphone of this board the operating frequency of sampling is Fs=2.4Mhz. Nexys 4 DDR runs at 100MHz so to produce 2.4MHz we built a clock divider.
3. **Decimation and Demodulation** The heart of processing on the FPGA is the CIC (Cascaded Integrator-Comb) filter. This block performs the function of digital demodulator:
    - Amplitude Extraction: Converts the bit density into a 24-bit PCM value in the time domain.
    - Anti-Aliasing Filtering: Uses a Sinc^2 type frequency response to eliminate high-frequency noise.
    - Downsampling: Reduces the frequency from the original 2.4 MHz to the target 48 kHz, enabling subsequent spectral analysis.
    - Bit Growth: The resolution increases from 1 bit (PDM) to approximately 25 bits (PCM) due to the cumulative structure of the CIC filter. The bit growth is defined by the relationship $Bmax{=}Bin{+}N\log_2(RM)$, which in our case ensures sufficient bit depth to maintain the quantization noise below the microphone's background noise floor.
    - FFT (Fast Fourier Transform);

1.2 Project constraints

In order to process analog vocal input data we need to consider some parameters constraints of digital communications for the vocal audio.

- **System operating frequency**: Nexys4DDR leverage it's internal clock with an operating frequency about 100MHz in order to handle complex computations efficiently.
- **Microphone operating system**: ADMP421 chip leverage a frequency of 2.4 MHz in order to capture better the information stored in the analog signal.
- **Oversampling and PDM**: This microphone samples at a frequency of 2.4 MHz the input data and apply an ADC conversion through PDM. It is used to convert the voltage of an analog signal into a single-bit pulse-density-modulated digital stream. A PDM bitstream is encoded from an analog signal through the process of a 1-bit delta-sigma modulation. This process uses a one-bit quantizer that produces either a 1 or 0 depending on the amplitude of the analog signal. A PDM modulator takes a continuous analog signal and converts it into a stream of bits (1s and 0s) at a very high frequency. The decision at each clock cycle is made according to the following cycle: The circuit tracks the difference (error) between the actual analog signal and the bit value it generated in the previous cycle. This error is accumulated (integrated) over time. In this way, every error successively influences every other quantization measurement and its error. This has the effect of averaging out the quantization error, while noise shaping it to push most of the quantization error into higher frequencies, which for audio signals would be ultrasonic.

  The Decision (The Comparator): At every clock cycle, a comparator checks the value at the integrator's output:
    - If the accumulated value is greater than 0: The comparator decides that the current instant is 1.
    - If the accumulated value is less than 0: The comparator decides that the current instant is 0.

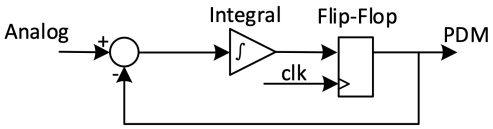


Figure 1: Basic building blocks of a CIC filter

To keep things simple, assume that the analog input and digital output have the same voltage range 0 Vdd. The input of the flip-flop acts like a comparator (any signal above Vdd/2 is considered as '1' and any input below Vdd/2 is considered '0'). The input of the integral circuit is the difference of the input analog signal and the PDM signal of the previous clock cycle. The integral circuit then integrates both of these inputs, and the output of the integral circuit is sampled by a D-Flip-flop.

Sum	Integrator Out	Flip-flop Output
0.4-0=0.4	0+0.4=0.4	0
0.4-0=0.4	0.4+0.4=0.8	1
0.4-1=-0.6	0.8-0.6=0.2	0
0.4-0=0.4	0.2+0.4=0.6	1
0.4-1=-0.6	0.6-0.6=0	0
0.4-0=0.4	0+0.4=0.4	0
0.4-0=0.4	0.4+0.4=0.8	1
0.4-1=-0.6	0.8-0.6=0.2	0

Figure 2: Sigma Delta Modulator with a 0.4Vdd input

- **PDM to PCM conversion** : In order to process digitally this PDM stream we apply a decimation filter called CIC (Cascaded Integrator-Comb), that takes this digital stream and converts it in a quantized signal at 24 bit x PDM sample in the time domain. CIC filter produces at the output a 24 bit downsampled signal with a rate R in order to slow down the stream. It's important to reduce the bit rate of the stream in order to increase the FFT resolution and reduce the size of the FFT.

$$FFTResolution = \frac{\text{downsample frequency}}{\text{FFT size}}$$

In this project we want to analyze the audible spectrum audio in high, mid, and low tones in the range of [0Hz,20000Hz].

To process this pass-band range we reduce the frequency of the signal from 2.4MHz to 48Khz with a decimation factor R =50.

$$R = \frac{\text{Microphone Operating Frequency}}{\text{FFT Operating Frequency}}$$

CIC filter has a sinc^2x frequency response that attenuate high frequency noise but introduce an amplitude attenuation in the audible spectrum about 5.2778dB.

- **Compensation filter**: CIC filter introduces an amplitude attenuation so to compensate this attenuation a FIR filter is designed in order to regenerate the original signal. Everything above 24 KHz is the Stopband so the FIR must attenuate as much as possible to prevent any residual noise that the CIC has not eliminated above the stopband from folding (aliasing) back and causing trouble in the passband. The compensating FIR filter is designed to operate in the 4 KHz transition band located between the limit of the audible band (20 kHz) and the Nyquist frequency of the decimated signal (24 kHz). This region allows the filter to have a roll-off (cutoff slope) that is physically realizable in hardware, avoiding the excessive computational complexity required by a brick-wall filter. The function of the FIR is twofold: to compensate for the attenuation introduced by the CIC in the passband and to ensure effective suppression of components beyond 24 kHz to prevent residual aliasing.

- **FFT**: in order to extracts high, mid, and low tones we need to analyze the digital signal in the frequency domain through FFT. FFT performs a transformation from the time domain to the frequency domain through the Xilinx FFT IP Core. The use of a rectangular window in time, while introducing spectral leakage, was preferred to optimize the hardware resources of the Nexys 4 DDR, maintaining a spectral resolution of 23.43 Hz, ideal for real-time visualization of the audio spectrum.

2 Filter Theory

2.1 CIC filter theory

As mentioned before, a cascaded integrator comb (CIC) filter is a multiplier free filter that has a low-pass response. These filters can also be used as a decimating filter. As the name suggests, a CIC filter is constructed by cascading two simple filter structures together: combs and integrators.



(a) Integrator Filter (b) Comb Filter

Figure 3: Basic building blocks of a CIC filter

The more integrator and comb filters that are cascaded together, the better the CIC filter does at filtering. The CIC filter exhibits better stopband attenuation, but the

sinc shaping in the passband is more pronounced and this causes amplitude attenuation.

In essence, a cascade of an integrator and a comb filter is equivalent to an FIR filter with an impulse response of a rectangular window. This translates to a sinc shaped filter response in the frequency domain. The general transfer function of the decimating CIC filter shown in Figure 4 is

$$H(z) = \left(\frac{1 - z^{-RM}}{1 - z^{-1}} \right)^N ,$$

and through Z transform that makes a conversion between discrete digital domain and complex z domain where z=e^{jw}, we arrive to the general frequency response of the CIC filter:

$$H(\omega) = \left(\frac{\sin\left(\frac{\omega RM}{2}\right)}{\sin\left(\frac{\omega}{2}\right)} \right)^N ,$$

.

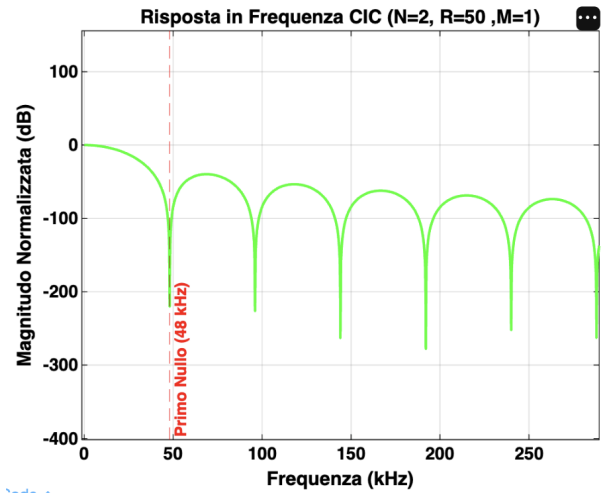


Figure 4: Frequency Response of CIC filter

Figure 4 shows the frequency response of CIC filters with one, three, and five stages to further illustrate this point. To obtain a decimation filter, N integrator filters are connected in series, followed by a subsampler with a decimation factor of R and N comb filters.

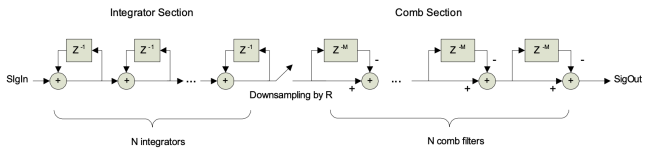


Figure 5: Block diagram of a CIC decimator using Xilinx LogiCORE™ IP CIC Compiler core that provides the ability to design and implement Cascaded Integrator-Comb (CIC) filters.

There are several advantages for using a CIC filter. The first reason is that they are multiplier free, making them ideal for high sample rate applications. Secondly, they can be organized such that they decimate the incoming signal while at the same time filtering with a low-pass filter to avoid aliasing in the frequency domain. One disadvantage of the CIC filter, however, is that they have relatively large gain. The gain of a CIC filter is $(RM)^N$. This leads to large accumulator registers in the integrator stages when fixed-point arithmetic is used. If the bit width of the accumulator registers is not sufficient to allow for the gain of the filter, they can overflow and cause the filter to be unstable. If the output width is $B_{\text{out}} = B_{\text{in}} + N\lceil \log_2(RM) \rceil$ where B_{out} is the bit width of the CIC filter output and B_{in} is the bit width of the input to the CIC filter, then the accumulators will not overflow and the filter will be stable.

Performing direct downsampling of the signal would result in catastrophic audio degradation due to the aliasing phenomenon. According to multi-rate sampling theory, during decimation, high-frequency components do not disappear but 'fold' back into the new baseband. Specifically, the noise energy that will overlap and corrupt the useful audio signal is that contained in bands centered around integer multiples of the new sampling frequency, namely the zones around 48 kHz, 96 kHz, 144 kHz, and so on.

The mathematical analysis of the frequency response shows that the CIC filter (N=2, R=50) has a -3 dB cutoff frequency located at approximately 15.28 kHz. This means that, while the filter is effective against aliasing at 48 kHz, it introduces natural attenuation in the upper part of the audible band (15-20 kHz) that requires subsequent correction.

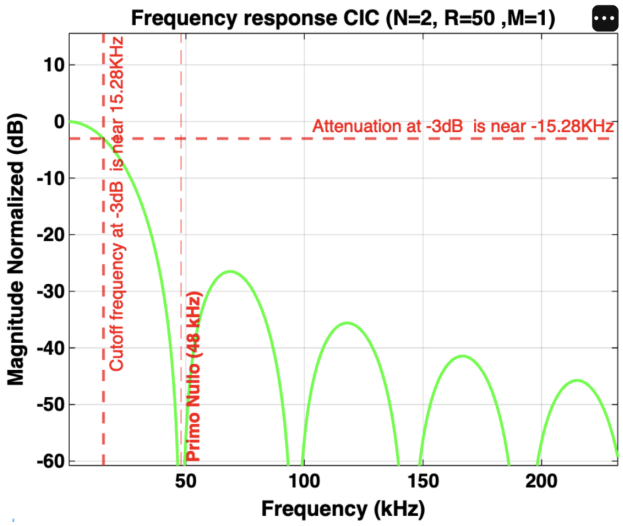


Figure 6: Cutoff frequency of the Integrator-Comb (CIC) filters.

As clearly evident from the figure 6 , the mathematical structure of the filter ensures that these nulls fall precisely at the exact multiples of the output frequency (as highlighted by the red line at the first null at 48 kHz). At these critical frequencies, the filter guarantees extremely deep attenuation (down to -220 dB). In summary, the CIC filter annihilates noise energy exactly within the folding

bands: when the system performs decimation, the aliasing that folds back into the audio band is, in fact, negligible, guaranteeing extremely clean signal conversion.

2.2 FIR filter theory

A digital filter is an algorithm designed to modify or extract information from a time-discrete signal. The goal may be to remove noise, emphasize certain frequencies, or even shape specific signal dynamics. The choice of filter type (FIR or IIR) often depends on specific requirements such as stability, phase linearity, computational complexity, and frequency response performance.

1. Key Features
    - Finite impulse response: The key characteristic of an FIR filter is that its impulse response tapers off after a finite number of samples. This means that, once the initial response ends, the effect of a past impulse fades away.
2. Design and Complexity
    - Simple design: FIR filters are often designed using techniques such as windowing or error-minimization optimization, ensuring precise control over the frequency response. High order: To achieve a strictly defined frequency response, FIR filters may require a high order (i.e., a large number of coefficients), thereby increasing the computational load.

3 FIlter design On MATLAB

3.1 CIC filter design

The design of the CIC filter leverages Matlab simulation tools that contains libraries for the configuration and simulation of the CIC filter designed. In order to replicate as similar as possible the CIC filter that appeared on Vivado with the CIC IP Core, it has been defined the value of the paramaters:

- R = 50
- M = 1;
- N = 2;
- Fs_{in} = 2400000Hz;
- fs_{audio}=48000 Hz;

$$R = \left(\frac{Fs_{in}}{fs_{audio}}\right)^N,$$

where:

- Fs_{in} is the input sampling frequency at which the signal is sampled at input of the CIC filter.
- fs_{audio} is the output frequency of the decimated audio signal.
- R is decimation factor and is set to 50 in order to obtain a signal with a frequency coherent with the standard sampling frequency for the audio
- M is the differential delay in the comb section stages of the filter and is set to 1 in order to reduce to minimum the latency during the decimation stage.
- N is the number of CIC stages and it is set to 2 in order to increase the attenuation in the side lobes and reduce the noise out of band and in the zero of the frequency response, in order to reduce the noise coupling at the nyquist limit frequency.

As we said before the passband droop also increases, thus narrowing the filter bandwidth. The increased droop might not be acceptable in some applications due to low accuracy of the estimation of the amplitude of the input audio signal, when we compute for instance FFT. Vivado documention suggests on setting a FIR filter in order to flatten the passband frequency response.

3.2 FIR filter design

With MATLAB it has been calculated the FIR filter that compensates the passband droop. Everything above 24 kHz is the Stopband. Here the FIR must attenuate as much as possible to prevent any residual noise that the CIC has not eliminated above the stopband from folding (aliasing) back and causing trouble in the passband.

The compensating FIR filter is designed to operate in the 4 kHz transition band located between the limit of the audible band (20 kHz) and the Nyquist frequency (f_{max}) of the decimated signal (24 kHz) where fs_{audio}=2(f_{max}) . This region allows the filter to have a roll-off (cutoff slope) that is physically realisable in hardware, avoiding the excessive computational complexity required by a brick-wall filter. The function of the FIR is twofold: to compensate for the attenuation introduced by the CIC in the passband and to ensure effective suppression of components beyond 24 kHz to prevent residual aliasing.

The CIC filter, while excellent for aliasing rejection, introduces a non-flat response in the audible range, known as passband droop. At 20 kHz, the signal is too attenuated.

The FIR filter equalizes this by applying an inverse gain profile, ensuring a flat response across the entire 0-20 kHz bandwidth.

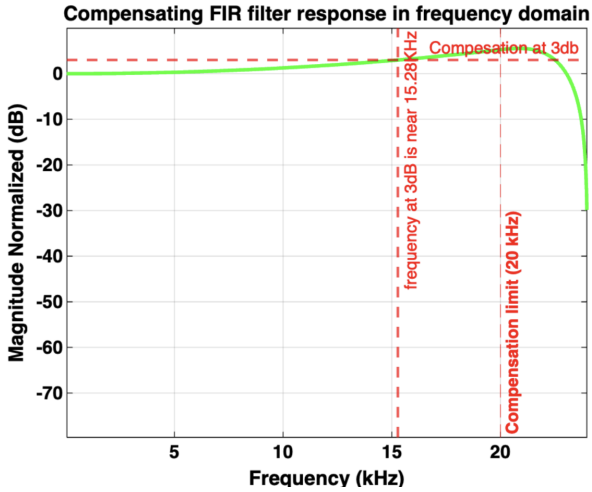


Figure 7: Compensating FIR filter response in frequency domain.

What is set in the FIR IP core are the coefficients of this filter that we can observe in the time domain with the graph below.

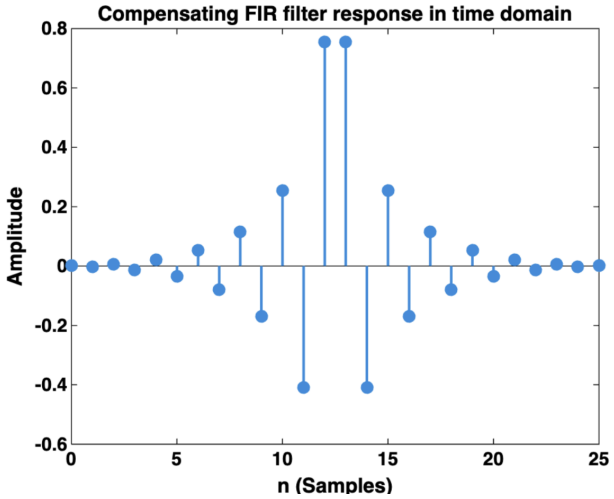


Figure 8: Compensating FIR filter response in time domain.

4 Data analysis on MatLab

4.1 Decimation

In order to see the effects of the decimation in the digital system we simulate the behaviour of the CIC filter on Matlab using different input analog signal with different frequencies. CIC filter samples data at 2.4MHz, therefore at the output is a reduce data stream sampled at 48Khz. CIC filter takes only one samples every 50 samples since we use a decimation factor of 50. In this way we obtain a data stream that runs at 48KHz with a reduced density of pulses in comparison with the original signal. This effects is shown the two figures below 9, 10.

4.2 FIR filter

After CIC filter the output signal from CIC is filtered in order to reduce and compensate the passband droop caused by CIC filter. In the time domain we see the worst and the best case clearly when we filter with FIR. In figure 9 the signal filtered by the FIR is compensated. In figure 10 the signal filtered by the FIR is completely attenuated and this is coherent with the frequency response of the FIR filter where at 24khz the signal is attenuated above 40dB reducing the amplitude of 100%. To prove that you must do calculations below.

$$\text{Gain (dB)} = 10 \cdot \log_{10}\left(\frac{P_{\text{out}}}{P_{\text{in}}}\right) \tag{1}$$

Substitute

$$P = \frac{V^2}{R} \tag{2}$$

Semplifying resistance R and collect exponent 2 we obtain:

$$\text{Gain (dB)} = 10 \cdot \log_{10}\left(\frac{V_{\text{out}}}{V_{\text{in}}}\right)^2 \tag{3}$$

$$\text{Gain (dB)} = 2 \cdot 10 \cdot \log_{10}\left(\frac{V_{\text{out}}}{V_{\text{in}}}\right) = 20 \cdot \log_{10}\left(\frac{V_{\text{out}}}{V_{\text{in}}}\right) \tag{4}$$

$$A_{\text{out}} = A_{\text{in}} \cdot \text{Attenuation}_{\text{CIC}}(fs) \cdot \text{Attenuation}_{\text{FIR}}(fs) \tag{5}$$

Where A_{out} is the final amplitude of the signal after FIR, A_{in} is the initial amplitude and fs is the frequency of the input singnal.

Note that FIR filter start the attenuation from 23Khz as we see fromm the FIR response and attenuate totally the frequencies above 24KHz.

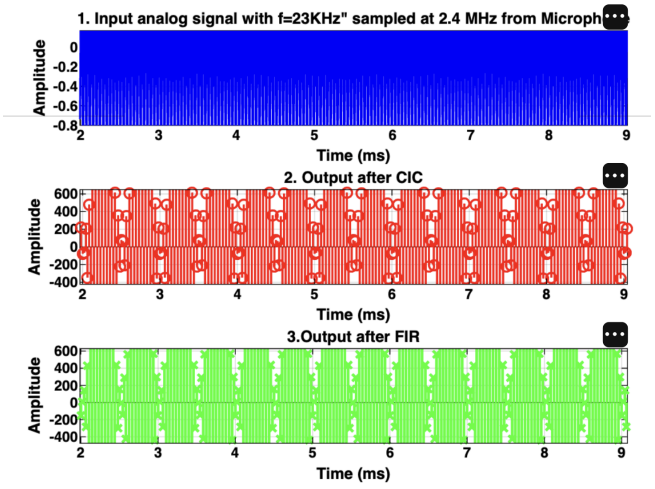


Figure 9: Compensating FIR filter response in time domain with test signal of 23KHz.

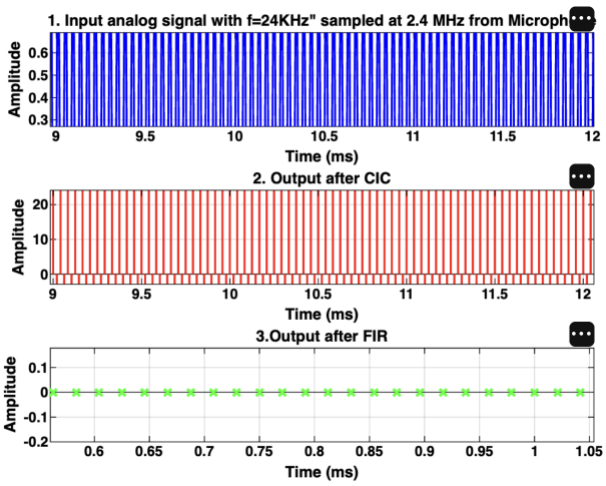


Figure 10: Compensating FIR filter response in time domain with test signal of 24KHz.

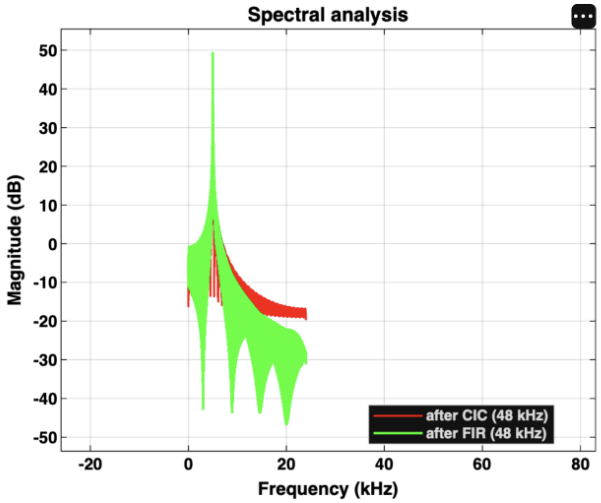


Figure 12: FFT after CIC and after FIR signal at 5KHz.

4.3 FFT analysis

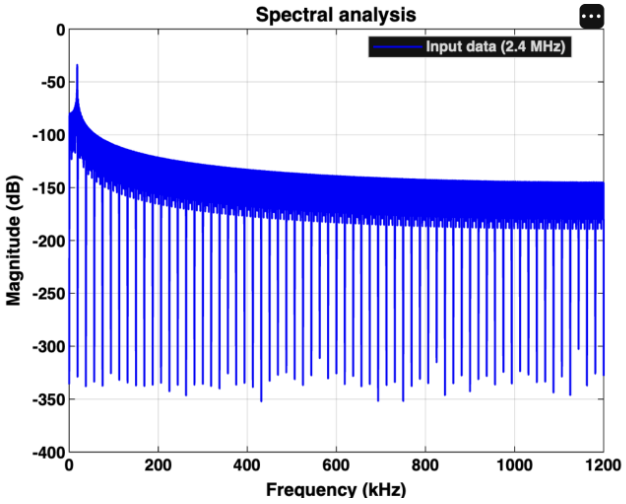


Figure 13: FFT input signal at 18KHz.

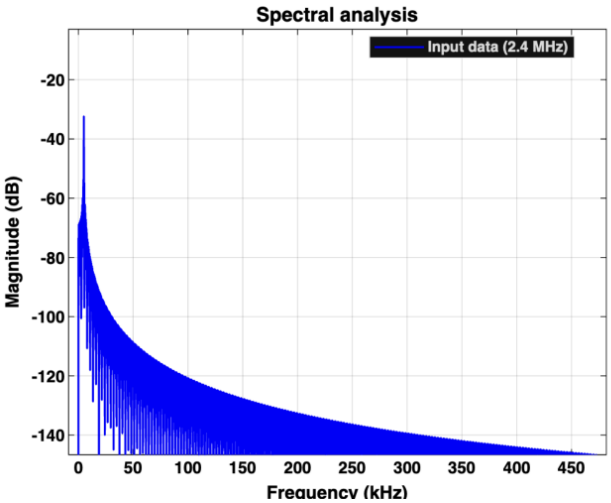


Figure 11: FFT input signal at 5KHz.

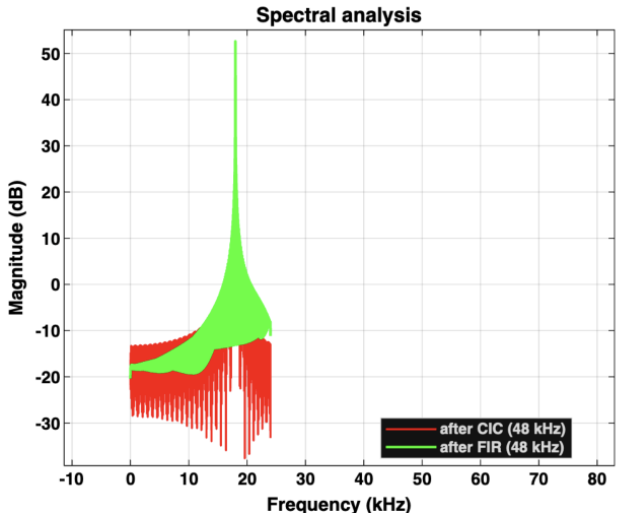


Figure 14: FFT after CIC and after FIR signal at 18KHz.

Note that the spectrum of the signal after CIC is amplified in amplitude about a factor of $(RM)^N$ where N corresponds to the number of CIC stages. In our case the maximum gain introduced by CIC filter is $(50*1)^2$ that is equal to 2500. This case is applied when the attenuation of the CIC is near 0KHz. If i use a frequency higher, the gain is reduced by a factor

$$A_{\text{riduzione}}(f) = \frac{|H_{\text{CIC}}(f)|}{R^N} = \left|\frac{\sin(\pi \cdot f \cdot R)}{R \cdot \sin(\pi \cdot f)}\right|^N \quad (6)$$

$$f = \frac{F_{\text{signal}}}{F_{s_in}} \quad (7)$$

$$F_{signal} = 2.4MHz, F_{s_in} \in [0, 24] \text{ kHz} \quad (8)$$

The value computed represents the reduction factor in percentage of the original amplitude when it goes through the CIC filter. To resume at 0KHz the gain is maximum at 24Khz the gain is minimum.

5 Block Design functionalities

5.1 Hardware Synchronization: The Processor System Reset Module

The `proc_sys_reset_0` IP core is integrated within the top-level block design to address a critical synchronization challenge inherent in synchronous digital systems: the proper alignment and distribution of the system reset signal.

5.2 Functional Necessity and the Metastability Problem

In high-frequency FPGA architectures, driving the reset inputs of sensitive DSP components (such as the CIC filter, FIR compiler, and FFT core) directly from an asynchronous physical source like the `CPU_RESET` pushbutton on the Nexys board presents a severe hazard.

An asynchronous de-assertion (release) of a reset signal can occur arbitrarily close to the active rising edge of the system clock. If the propagation delay places the transition within the critical setup (t_{su}) or hold (t_h) time windows of the internal flip-flops, the affected registers can enter a state of **metastability**. This hardware failure causes unpredictable logic states, corrupts the accumulator stages of the decimation filters, and halts the downstream pipeline of the spectrum visualizer.

5.3 Architectural Implementation: Asynchronous Assert, Synchronous De-assert

To mitigate these timing violations, the `proc_sys_reset` module implements a deterministic synchronization strategy known as *Asynchronous Assert, Synchronous De-assert*. The operational workflow is structured as follows:

- **Input Interface:** The core continuously monitors the raw, asynchronous external reset signal (`ext_reset_in`) alongside the reference system clock (`slowest_sync_clk`), which operates at 100 MHz.
- **Internal Multi-stage Synchronization:** Upon pressing the physical button, the reset signal is asserted immediately and asynchronously to force all registers into a known valid state without delay. However, when the button is released, the core passes the transition through an internal multi-stage flip-flop synchronizer chain. This chain shifts the de-assertion event, guaranteeing that the reset is dropped in strict alignment with the active edge of the clock.
- **Output Distribution:** The core generates a fully synchronized, glitch-free active-low peripheral reset output (`peripheral_aresetn`).

5.4 Impact on the AXI-Stream Data Path

The primary objective of this architecture is to eliminate *reset skew* across the physical fabric of the FPGA. By ensuring that the `peripheral_aresetn` line changes state globally and simultaneously, every component within the audio processing chain emerges from the initialization state during the exact same clock cycle. This uniform startup behavior is vital for the AXI-Stream protocol, as it prevents data mismatches, aligns the initial handshaking flags (`tvalid`/`tready`), and ensures the coherent streaming of the processed audio samples.

5.5 Analysis of the IP Reset Configuration Parameters

As shown in the IP Core configuration, the module is configured with an `Ext Reset Logic Level` equal to `0`, in perfect compliance with the electrical behavior of the CPU reset button on the Nexys board (active-low). The `Ext Reset Active Width` parameter is set to `4` clock cycles; this configuration acts as a hardware filter against fast transients and impulsive noise (*glitches*), ensuring that the system reset is propagated only upon a stable and prolonged press on the physical component.

5.6 Clock Divider

This clock divider takes as input the 100 MHz clock provided by the E3 pin on the Nexys4DDR and provides the clock for the microphone using synchronous logic that generates rectangular pulses at a frequency of 2.4 MHz. This 2.4 MHz signal acts as the operational clock for the microphone, meaning that the sampling frequency of the microphone is exactly 2.4 MHz.

5.7 PDM wrapper (axi stream adapter)

This wrapper is very important because, if we want to decimate the data from the microphone, we need to wrap the single bit that comes from the microphone following this logic:

- **Bit 1**: is considered in CA2 a positive value that wrapped in 8 bits is equal to 00000001
- **Bit 0**: is considered in CA2 a negative value that wrapped in 8 bits is equal to 11111111

  This entity is driven by the 2.4MHz clock and at the output produces the wrapped input bit, that can be sent to the CIC filter. Note that CIC filter accepts a mimimum size input of 8 bit.

  To guarantee data, validity when a new sample arrives at the wrapper, the `tvalid` flag is asserted only when the block outputs the newly wrapped 8-bit sample. This signaling mechanism prevents the downstream block (the CIC filter) from processing false or redundant data during clock cycles when no new microphone sample is actually present. The inclusion of the `m_lrsel` signal directly within the wrapper entity represents a deliberate architectural choice . It is crucial because, by setting this signal to `'0'` inside the entity, the microphone is configured to operate on its Left channel, instructing it to transmit data exclusively on the **falling edge** of the 2.4 MHz clock. This logic is connected to a physical pin on Nexys4DDR called F5.

5.8 CIC filter

This CIC filter as we said before in the CIC filter theory, downsaples the data stream that comes from the wrapper with a decimation factor of 50. The output data from the CIC is a data that have a size of 16 bits considering the fact that CIC filter has an internal gain in terms of output bit width. By leveraging the configuration options of the Xilinx CIC Compiler IP core, the input width has been successfully minimized to `2` bits, significantly optimizing the number of routing connections and fabric logic utilization within the FPGA. Since the essential two's complement bipolar information (+1 and −1) can be completely and uniquely represented using only 2 bits (01_2 and 11_2), we can safely discard the 6 higher-order redundant bits from the wrapper's 8-bit output bus, retaining only the 2 least significant bits (LSBs) without any loss of signal integrity or data precision. In a similar manner to the frontend wrapper, when the CIC Compiler processes the incoming PDM data stream, it asserts its output `m_axis_data_tvalid` flag. This handshake signal is essential to guarantee data validity along the downstream datapath; since the CIC filter performs sample rate decimation, the `tvalid` acts as a qualified clock enable for the subsequent FIR filter, ensuring it only processes newly computed downsampled blocks, ignoring invalid data cycles.

5.9 FIR filter

CIC filter is followed by FIR filter that as we said in the FIR filter theory compensates the passband droop and cut all the frequency beyond 24KHz. FIR filter receives the data processed by the CIC and outputs a data with a size of 24 bit. To guarantee absolute data validity when a new sample arrives at the subsequent FFT acceleration block, the `m_axis_data_tvalid` flag is strictly asserted only when the FIR filter completes its internal convolution and outputs the newly processed data. This synchronous handshaking mechanism prevents the FFT core from reading duplicated or unaligned samples, ensuring that the spectral analysis is executed exclusively on coherent, fresh time-domain frames.

6 IP Core Parameter Configuration: FIR Compiler

The `fir_compiler_0` module was configured within the Vivado environment by setting its arithmetic and geometric parameters to precise specifications. This approach aims to preserve audio signal fidelity while minimizing the implementation footprint on the FPGA fabric.

6.1 Detailed Implementation Options

The configuration choices regarding filter coefficients, input data, and output data are structured as follows:

- **Coefficient Options:**
  - **Type:** `Signed`. It handles both positive and negative coefficients, which is an indispensable requirement to map the mathematical response of a digital low-pass filter
  - **Quantization:** `Maximize Dynamic Range`. It automatically optimizes the resolution of the ideal weights, scaling them to fully occupy the available binary grid and minimizing coefficient quantization noise.
  - **Width:** `24 bits`. It represents the bit precision selected to map the quantization of the filter coefficients.
  - **Fractional Bits:** `23 bits`. With the maximum precision mode enabled, the tool automatically allocates 1 bit for the sign and 23 bits for the fractional part.
- **Input Data Options:**
  - **Type:** `Signed`. The incoming digital audio signal is correctly processed in two's complement format
  - **Width:** `16 bits`. It defines the width of the input bus, matching perfectly with the output interface of the upstream CIC filter.

  - **Fractional Bits:** `0 bits`. The input audio samples are interpreted as pure integer numbers.

- **Output Data Options:**
  - **Rounding Mode:** `Convergent Rounding to Even`. It performs statistical rounding to the nearest even number. This choice eliminates the introduction of a parasitic DC offset induced by standard truncation arithmetic.
  - **Width:** `24 bits`. The final output bus directed toward the FFT accelerator features a total precision of 24 bits .
  - **Fractional Bits:** `5 bits`. The output incorporates 5 fractional bits due to internal bit growth during the convolution's multiply-accumulate operations.

- **Filter Structure:**
  - **Structure Type:** `Inferred`. The compiler autonomously analyzes the coefficient array at synthesis time to deduce the optimal hardware topology .

6.2 Architectural Design Justifications

The proposed configuration is justified by three fundamental engineering considerations:

1. **Coefficient Optimization:** Adopting the maximum dynamic range paired with 23 fractional bits allows the system to closely approximate the ideal filter frequency response. This minimizes the effects of coefficient quantization noise, thereby preserving the passband selectivity and linearity of the audio spectrum .

2. **Convergent Rounding Selection:** Utilizing *Convergent Rounding to Even* is highly preferred over standard truncation or asymmetric rounding. This approach cancels out the introduction of a parasitic DC bias distortion generated within the hardware multiplication cascades .

3. **Inferred Structure Efficiency:** By setting the filter structure to `Inferred`, Vivado is tasked with analyzing the linear symmetry of the low-pass filter . If the coefficients exhibit symmetry, the tool leverages the hardwired *Pre-Adder* architecture embedded within the Nexys board's physical `DSP48E1` blocks. This effectively halves the number of logical multipliers required on silicon, drastically reducing the area utilization on the chip .

6.3 FFT IP core

FFT is performed using the FFT ip core optimized for complex calculations and large amount of data. The size of the FFT transform is set to 2048 samples. FFT starts processing data when the tvalid that runs at 48KHz from FIR is asserted. A input signal TLAST is connected to the output of the binary counter ip core called THRESH: this counter binary runs at 100MHz like FFT and it is enabled by the CE signal connected to the tvalid output of the FIR: when a valid data is sent to FFT this counter at the rising edge of 100MHz clock increases the threshold of 1 every time a tvalid signal from FIR is asserted. In this way the number of samples is syncronous with the arrival time of the data at the FFT input. FFT running at 100MHz and receiving the data at 48khz has 2083 Clock cyclce at 100MHz to perform calculations and this very safe in term of time computational margins. The FFT IP core provides some settings for optimization: The architecture chosen is Pipeline, Streaming I/O. The Pipelined Streaming I/O solution pipelines several Radix-2 butterfly processing engines to offer continuous data processing. Each processing engine has its own memory banks to store the input and intermediate data. The core has the ability to simultaneously perform transform calculations on the current frame of data, load input data for the next frame of data, and unload the results of the previous frame of data. You can continuously stream in data and after the calculation latency, can continuously unload the results. The throughput of the FFT core is rated at 100 MSPS, driven by a 100 MHz system clock which establishes a discrete clock period of 10 ns. In this pipelined architecture, this 10 ns interval represents the time required for the processing data frame to traverse a single computational stage. Since the 2048-point Radix-2 algorithm is spatially distributed across $\log_2(2048) = 11$ cascaded stages, a complete data frame (2048 samples) requires a total structural latency of 62.629 us to propagate from the input registers to the final output.

6.4 FFT IP core: introduction to Arithmetic configuration

The computational architecture of the `xfft_0` module, operating in *Pipelined Streaming I/O* mode, requires a rigorous definition of numerical precision and quantization strategies at each stage of the pipeline. In the implemented design, the core has been configured by combining the **Unscaled** scaling option and the **Truncation** rounding mode. This combination represents a precise engineering compromise (*trade-off*) aimed at maximizing signal accuracy in the frequency domain, while simultaneously minimizing logic area occupancy and propagation latencies on the FPGA silicon.

6.4.1 The Unscaled Arithmetic Mode and Bit Growth Management

From a mathematical standpoint, the 2048-point ($N = 2048$) FFT algorithm is structured across $\log_2(N) = 11$ cascaded complex computational stages. Within each stage, the elementary *Radix-2 Butterfly* units combine the input samples through complex addition and subtraction operations, alternating with multiplications by the rotation coefficients (*Phase Factors*). Each addition opera-

tion inherently introduces the possibility of an increase in the maximum value of the data, a phenomenon known as *integer bit growth*.

By selecting the **Unscaled** option, the IP Core does not apply any intermediate scaling or arithmetic division operations (such as right-shifting the bitstream) between the different stages of the pipeline. Consequently, the entire bit growth is preserved and dynamically carried through to the output bus (`m_axis_data_tdata`). Although this mode progressively widens the *datapath*, thereby requiring larger internal registers, it guarantees the absolute absence of overflow phenomena and preserves the integrity of the system's dynamic gain. Regarding the audio signal, the absence of intermediate attenuations maintains a high signal-to-noise ratio (SNR), allowing the system to discriminate with extreme precision even the lowest amplitude harmonic components, which are essential for a correct mapping of high frequencies.

6.4.2 The Truncation Rounding Mode and Critical Path Optimization

At each stage of the *Pipelined Streaming* structure, multiplications by the trigonometric phase factors generate a word-length expansion toward the least significant bits (*LSBs*), introducing a fractional part that needs to be discarded (*LSBs trimming*) to prevent an uncontrolled explosion of the bus width.

The choice of the **Truncation** rounding mode forces the circuit to directly eliminate the excess least significant bits, performing a systematic rounding downward (the mathematical *floor* function). From a purely theoretical perspective, truncation introduces a non-zero mean quantization error equal to $-0.5\,\mathrm{LSB}$, which results in a constant micro-offset known as a negative *DC bias*. The manufacturer's official technical documentation highlights that this distortion could be eliminated by adopting *Convergent Rounding* (Round-to-Even), an *unbiased* scheme capable of statistically compensating for rounding errors.

However, the hardware implementation of *Convergent Rounding* would require the insertion of auxiliary combinational logic and adder circuits at the output of every single butterfly in the pipeline to evaluate the parity of the preceding bit. Such additional logic would lead to a significant increase in look-up table and flip-flop allocation (*increases slice usage*) and would introduce extra delay stages into the critical path, penalizing the overall transform time (*increase in transform time due to additional latency*).

In the context of the present project, the adoption of truncation responds to a strict logic of hardware optimization:

- It allows for the elimination of area overhead, maintaining the dynamic consumption of the FPGA within the minimal target of `0.455 W`.
- It reduces the combinational delays of the pipeline, ensuring full compliance with timing constraints and maintaining a wide *Worst Negative Slack* margin (WNS = `+0.623 ns`) at the operational frequency of `100 MHz`.

The potential degradation of the SNR induced by the *DC bias* of the truncation is completely negligible at the system level: the simultaneous **Unscaled** configuration maintains an intrinsic resolution high enough to confine the quantization error to the lowest bits of the bus. This infinitesimal offset is practically invisible on the screen because it represents an error on a single pixel scale and is completely negligible.

6.4.3 Lower Bit Selection and Geometric Impact on the Display

In the interface module toward the video backend, the mapping of the spectrum amplitude onto the vertical resolution of the VGA display (equal to 480 pixels) was performed by extracting the 9 bits that contains more information in terms of energy in the data bus outputting from the Energy computation blocks.

From a numerical analysis standpoint, the choice to isolate the lower portion of the data bus meets the need to preserve the maximum fractional resolution and the sensitivity of the algorithm to low-dynamic baseband signals. In an *Unscaled* architecture, although the theoretical word growth develops toward the upper bits (*MSBs*) to prevent overflow, the low-energy harmonic components remain confined within the lower quantization levels.

However, the resulting systematic translation corresponds geometrically to an infinitesimal fraction of the total screen height (equivalent to approximately a single pixel out of the 480 available). This constant displacement does not affect the linearity of the visualization nor does it introduce detectable harmonic distortion, proving entirely transparent to the observer and validating the effectiveness of the compromise between fractional accuracy and logic area savings on the FPGA. Note that the composite input vector driven into the `s_axis_data_tdata` port of the FFT compiler allocates the 24-bit signed real-valued audio sample from the FIR filter within its lower 24 bits ([23:0]), while the upper 24 bits ([47:24]) are hardwired to a null vector to represent the imaginary component.

6.5 Configuration and Hardware Architecture of the FFT Compiler IP Core

Spectral processing of the audio signal is handled by the `FFT Compiler` IP Core (module `xfft_0`), configured to compute the discrete Fourier transform over a frame width of $N = 2048$ points. From a system perspective, the integration of this hardware accelerator requires a careful balance between the FPGA's timing constraints, the management of synchronous data flows, and the optimization of the allocated silicon area.

6.5.1 Choice of Hardware Architecture: Pipelined Streaming I/O

For the `xfft_0` module, the **Pipelined, Streaming I/O** architecture was selected, a configuration that differs structurally from *Burst*-type implementations. This architecture unfolds the FFT algorithm on the silicon, implementing a cascade of independent computation stages, each equipped with elementary *Radix-2 Butterfly* arithmetic units and dedicated memory registers.

The adoption of this structure has specific implications in terms of *Digital Design*:

- **Continuous Data Streaming e Throughput:** The architecture is optimized to ensure a continuous *Target Data Throughput* of `100 MSPS`. The core allows for the simultaneous and uninterrupted feeding of input samples and the submission of new 2048-point frames without computational downtime or phase-shift latencies (*no latency overhead* between frames), since the various stages operate in a pipeline.

- **AXI-Stream Interconnection and Frame Synchronization:** Since the FFT operates in continuous streaming at the system clock frequency of `100 MHz` but receives data at the FIR filter's event rate (48 kHz), frame delimitation requires deterministic synchronization. This is handled by the `s_axis_data_tlast` signal. A `Binary Counter` with the *Threshold* set to `2048` is configured with the *Clock Enable* (`CE`) input driven by the FIR's `tvalid` signal. In this way, the register increments only at each actual valid audio sample, and the *Threshold* pulse sets the `tlast` flag strictly simultaneously with the first data of the new frame, eliminating the risk of premature truncation or corruption of the internal buffer of the FFT stages.

6.5.2 Arithmetic Configuration: Analysis of Unscaled and Truncation Parameters

The module operates in *Fixed-Point* numeric format, combining the *Scaling* option of type **Unscaled** with the *Rounding Mode* of type **Truncation** to define the numerical precision along the pipeline.

From a mathematical standpoint, the 2048-point algorithm is organized into $\log_2(N) = 11$ cascaded stages. Each complex addition operation within the butterflies introduces the possibility of an increase in the maximum data value (*integer bit growth*). When the **Unscaled** option is selected, the IP Core does not apply any scaling or right-shifting operations between the different stages. The entire bit growth is preserved all the way to the output bus (`m_axis_data_tdata`), ensuring the absolute absence of overflow and maintaining a high signal-to-noise ratio (SNR), ideal for discriminating very low-amplitude harmonic components.

To handle the least significant bits (*LSBs trimming*) at the output of the rotations, the **Truncation** mode was chosen, which directly eliminates excess bits by systematically rounding down. From a theoretical standpoint, truncation introduces a non-zero average quantization error of -0.5 LSB, which results in a constant micro-offset known as a negative *DC bias*. Implementing *Convergent Rounding* would have eliminated this distortion, but it would have required auxiliary combinational logic and summing circuits at each stage to evaluate the parity of the previous bit. This excess hardware would have resulted in increased logic slice usage and introduced additional delay stages in the critical path.

The adoption of truncation therefore eliminates this area overhead, keeping the FPGA's overall dynamic power consumption within the minimum target of `0.455 W` and ensuring full compliance with timing constraints with a generous *Worst Negative Slack* margin (WNS = `+0.623 ns`). The SNR degradation caused by the *DC bias* has no impact at the system level, as it is masked by the high dynamic range maintained by the *Unscaled* configuration.

6.5.3 Optimization of Complex Multipliers Using a Three-Multiplier Structure

The implementation of complex multipliers, which perform calculations between samples and trigonometric rotation factors (*Phase Factors*), utilizes the **Use 3-multiplier structure** architecture, designed for *resource optimization*. The traditional multiplication of two complex numbers $(A + jB) \times (C + jD)$ requires four real multiplications and two real sums: $\text{Re} = AC - BD$ and $\text{Im} = AD + BC$. The optimized 3-multiplier structure, on the other hand, introduces three intermediate real products, defined as follows:

$$P_1 = A \times (C + D) = AC + AD \tag{9}$$

$$P_2 = (A + B) \times D = AD + BD \tag{10}$$

$$P_3 = (B - A) \times C = BC - AC \tag{11}$$

The core architecture synthesizes the outputs by applying only post-addition and post-subtraction operations on the three calculated coefficients. The real part is reconstructed via the subtraction $P_1 - P_2$:

$$P_1 - P_2 = (AC + AD) - (AD + BD) \tag{12}$$

$$= AC + AD - AD - BD = AC - BD \tag{13}$$

The mixed terms AD cancel each other out, yielding the exact real component. Similarly, the imaginary component is reconstructed through the sum $P_1 + P_3$:

$$P_1 + P_3 = (AC + AD) + (BC - AC) \tag{14}$$

$$= AC + AD + BC - AC = AD + BC \tag{15}$$

In this case, the terms AC cancel each other out, isolating the expression for the imaginary part.

This mathematical framework allows us to convert a high-cost hardware resource (a full silicon-level multiplier) into extremely economical addition operations. Since the

pre-additions $(C + D)$, $(A + B)$, and $(B - A)$ are fed directly into the *Pre-Adder* modules integrated within Xilinx's `DSP48E1` hardware primitives, the algorithmic overhead is absorbed with zero latency within the same silicon block, reducing the total number of DSP slices required by 25%.

6.5.4 Boundary Conditions for Real Input and Propagation in the Complex Domain

The IP core's input channel `s_axis_data_tdata` receives a composite vector in which the 24 bits of fixed-point data, coming from the FIR filter, occupy the lower section dedicated to the real part (Re = $[23 : 0]$). At the same time, the 24 bits of the upper section ($[47 : 24]$), reserved for the imaginary part (Im), are statically set to a zero vector. This direct manipulation via bus concatenation is unique to *Fixed-Point* arithmetic, in which the position of the decimal point is implicitly defined by the hardware. This approach allows for native integration of the blocks without resorting to active conversion modules for the IEEE-754 standard (which are necessary in *Floating-Point* configurations), reducing the logical interface overhead to zero.

Setting the initial imaginary component to zero ($B = 0$) results in an immediate algebraic simplification exclusively in the first computation stage of the algorithm (*Stage 0*). In this section, the elimination of the term B reduces the intermediate products of the butterfly to $P_2 = A \times D$ and $P_3 = (-A) \times C$. However, the persistence of the hardware architecture with three complex multipliers along the entire *datapath* of the pipeline remains a necessary design constraint.

The FFT algorithm operates by projecting the time-domain signal into the frequency domain through successive multiplications by trigonometric rotation factors (*Twiddle Factors*). Although the input signal to *Stage 0* is purely real, its linear combination and subsequent complex rotation ($C + jD = \cos\theta - j\sin\theta$) instantly generate, at the output of the first stage, partial vectors with dynamic, non-zero real and imaginary components. In fact, the algorithm projects the time signal by multiplying it by the first trigonometric rotation factors ($W_N = C + jD = \cos\theta - j\sin\theta$). Implementing this operation on silicon, the product of the real partial sample and the complex coefficient generates a combined output vector equal to:

$$V_{out_stage0} = (A \times C) + j(A \times D) \tag{16}$$

However, the fundamental transition occurs at the exact moment the signal **exits** *Stage 0*.

The data stored in the pipeline registers at the end of the first stage is no longer a pure real number: it has a new real component (AC) and a new dynamic, nonzero imaginary component (AD). For the subsequent $\log_2(N) - 1 = 10$ stages of cascaded computation, the environment variables A and B assume entirely complex configurations ($B \neq 0$). Maintaining a consistent structural layout with three multipliers per pipeline stage ensures perfect symmetry in propagation times and maximum regularity in signal routing on the Artix-7 FPGA silicon. This architectural continuity extends the effectiveness of the 25% savings in DSP resources to the entire transformation module, ensuring a *throughput* of `100 MSPS` and the stability of system timings.

6.6 Spectral Energy Computation and Bit-Slicing Architecture

The output interface of the `xfft_0` IP Core provides the computed spectral coefficients via a single unified AXI4-Stream data bus, `m_axis_data_tdata`, with a total width of 80 bits. Within this bus, Xilinx packs both orthogonal components of the complex frequency domain signal:

- **Upper 40 bits (`tdata[79:40]`):** Represent the Imaginary component (Im).
- **Lower 40 bits (`tdata[39:0]`):** Represent the Real component (Re).

To process this data, a dedicated hardware demultiplexing structure is implemented. Two hardware *Slice* modules are utilized to isolate the independent 40-bit channels synchronously.

Energy Spectrum Optimization To determine the magnitude of the spectral bins for visualization purposes, computing the standard Euclidean norm $|X(k)| = \sqrt{\mathrm{Re}^2 + \mathrm{Im}^2}^2$ would introduce significant structural overhead. The square root operation requires iterative CORDIC architectures or large combinational logic blocks, penalizing both system latency and critical path timing.

To maximize hardware efficiency, the system calculates the **Energy Spectrum** (or magnitude squared) instead:

$$|X(k)|^2 = \mathrm{Re}^2 + \mathrm{Im}^2 \tag{17}$$

This mathematical simplification is mapped onto the FPGA fabric using two parallel multipliers to calculate Re^2 and Im^2 respectively, followed by a high-speed pipelined adder that computes their sum. This approach preserves the proportional relationship of the audio frequencies, which is ideal for real-time VGA rendering, while drastically lowering LUT consumption and maintaining a safe timing slack at `100 MHz`.

6.7 Spectrum Isolator and Synchronous Memory Addressing

To interface the high-speed streaming output of the energy computation stage with the video frame-buffer, a dedicated *Spectrum Isolator* module is implemented. This entity is responsible for data throttling, spectral windowing, and synchronous BRAM address generation based on the physical properties of the processed signal.

Symmetry Optimization and Windowing Since the initial audio input is a purely real-valued time-domain signal, its Discrete Fourier Transform (DFT) inherently exhibits Hermitian symmetry. Consequently, the power spectrum magnitudes of the upper half bins ($k \in [1024, 2047]$) are a mirror image of the lower half bins ($k \in [0, 1023]$). To maximize hardware resource efficiency and eliminate redundant memory allocations, the *Spectrum Isolator* discards the upper redundant spectrum, selectively capturing only the first 1024 meaningful frequency bins representing the unique spectral content up to the Nyquist frequency ($\frac{f_s}{2}$).

Pipeline Alignment via Shift Registers The combinational and sequential logic mapped onto the DSP slices for the energy computation ($Re^2 + Im^2$) introduces a deterministic structural latency of exactly **13 clock cycles** at `100 MHz`. To prevent spatial-temporal mismatch between the computed data bus and its respective AXI4-Stream control signals, the `tvalid` and `tlast` flags are routed through a digital pipeline delay line implemented via a Xilinx *Shift Register* IP Core configured to an identical depth of 13 stages. This guarantees perfect phase alignment at the input boundaries of the isolator.

Addressing Logic and Capture Gating The internal control logic operates as a synchronous finite state machine triggered by the delayed `tvalid` strobe:

- Upon the assertion of the synchronized `tvalid`, an internal address counter dedicated to the dual-port BRAM is incremented sequentially from 0 to 1023.
- A local gating signal, `capture_en` (Write Enable), is asserted high (`'1'`) strictly for the duration of the first 1024 incoming samples to allow memory populating.
- When the internal counter reaches the terminal index value of 1023, `capture_en` de-asserts to `'0'`, effectively masking out the remaining 1024 symmetrical samples from being written.
- The assertion of the delayed `tlast` signal indicates the formal boundaries of the complete 2048-point FFT packet, resetting the internal address counter and arming the system for the subsequent audio frame initialization.

6.8 DiSplay the FFT spectrum via VGA signal

DUAL PORT BRAM To store the data that need to be plotted on the display via VGA it has been used a DUAL PORT BRAM that contains 1024 samples that occupies each 81 bits. The Porta A runs at 100MHz and receives the adresses driven by `ram_addr` signal, that acts as an index which drives correctely the date in the correct position in the BRAM. Port B runs at 25MHz that is standard clock for VGA signal. THIS BRAM is shared, which means that you can put data from port A and read data from port B. The adresses where the bram has to pick the value of energy are sent by the VGA display through `ram_addr`, which is calculated using an index that varies from 0 to 1024 so we have 32 bars, each with a resolution of 23.4375Hz. In order to capture data from 0 to a frequency near 20KHz the adresses of the BRAM follow this relationship see Section 7.9.

6.9 Frequency Mapping and BRAM Addressing

The magnitude data calculated by the FFT module are stored in a block memory (Block RAM or BRAM). Each visual channel of the spectrum analyzer, identified by the bar index N (where $0 \leq N \leq 31$), must sample a specific frequency bin.

The geometric-mathematical relationship linking the bar index N to the physical read address of the memory (`bram_addr`) is defined by a constant scale factor of 27:

$$\texttt{bram_addr} = N \times 27 \quad \text{where} \quad N \in [0, 31] \tag{18}$$

To convert this logical address into the corresponding actual physical frequency value expressed in Hertz (Hz), the system's frequency resolution (Δf) must be taken into account. In this specific case, the spacing between two consecutive spectral bins is equal to:

$$\Delta f = \frac{f_s}{N_{\text{FFT}}} = 23.4375 \text{ Hz} \tag{19}$$

Therefore, the center physical frequency f_N associated with each of the 32 bars of the spectrogram is determined by combining the BRAM addressing step with the spectral resolution, according to the formula:

$$f_N = (N \times 27) \times 23.4375 \text{ Hz} = \texttt{bram_addr} \times 23.4375 \text{ Hz} \tag{20}$$

By multiplying the read address by the conversion factor 23.4375, the hardware performs a linear mapping of the acoustic spectrum. For example, the first bar ($N = 0$, `bram_addr` $= 0$) corresponds to the DC component of the signal (0 Hz), while the last bar ($N = 31$, `bram_addr` $= 837$) maps the spectral component centered at:

$$f_{31} = 837 \times 23.4375 \text{ Hz} = 19617.1875 \text{ Hz} \approx 19.6 \text{ kHz} \tag{21}$$

This covers almost the entire spectrum of frequencies audible to the human ear.

VGA TIMING This entity operates at 25 MHz and drive the HS (Horizontal Sync) and VS (Vertical Sync) signals, which are important for driving correctely VGA signal. In addition to these two display control signals, these entities provide the signals that generate the color for each pixel and the condition for plotting the bars—namely, the 'videoactive' flag set to 1, which indicates that you are within the monitor's boundaries.

STABILIZATION OF THE `pulse_IN` SIGNAL To capture the `pulse_IN` signal, this entity acts as a safety bridge between two different circuit speeds. It takes the 'Frame Ready' signal originating in the fast FFT domain (at 100 MHz) and translates it into a stable and visible pulse for the slower VGA domain (at 25 MHz). This prevents the monitor from dropping frames and eliminates any electrical instability during the transition." To stabilize the signal, it uses a 2-Flip-Flop Synchronizer algorithm (Double-Register Synchronizer) to safely cross the clock domains and eliminate any risk of electrical metastability."

7 Module VGA_Display

The `VGA_Display` module represents the final visualization block of the processing chain. Its purpose is to convert the 1024 spectral magnitude values stored in the BRAM into RGB video signals compatible with the VGA standard (640 × 480 pixels at 60 Hz), producing on the screen a real-time audio spectrum analyzer with 32 colored vertical bars and dynamic peak indicators.

7.1 General Architecture

The module operates entirely in the VGA clock domain at 25 MHz and receives three categories of signals:

- The current pixel coordinates (`pixel_x`, `pixel_y`) and the `video_active` signal from the `VGA_Timing` module;
- The 81-bit magnitude data (`bram_data`) read from port B of the `Spectrum_BRAM`, addressed via `bram_addr`;
- The `frame_ready` signal from the `Pulse_CDC` module, which signals the availability of a new spectral frame.

The module operation is structured in three distinct phases that cooperate to produce the final visual animation.

7.2 Phase 1 — Geometric Screen Mapping

The 640-pixel wide screen is divided into 32 spectral channels. Each channel is allocated 20 pixels of total width, of which 16 pixels constitute the visible body of the bar and 4 pixels form the black separation gap between adjacent bars:

$$\text{bar_index} = \left\lfloor \frac{px}{20} \right\rfloor, \qquad \text{in_bar} = \begin{cases} 1 & \text{if } (px \bmod 20) < 16 \\ 0 & \text{otherwise} \end{cases} \tag{22}$$

Each bar is associated with an FFT bin through the relation:

$$\text{bram_addr} = \text{bar_index} \times 27 \tag{23}$$

With a sampling frequency of 48 kHz (obtained by decimating the PDM signal at 2.4 MHz by a factor of 50 in the CIC filter) and a 2048-point FFT, the spectral resolution per bin is:

$$\Delta f = \frac{F_s}{N_{FFT}} = \frac{48000}{2048} \approx 23.4 \text{ Hz/bin} \tag{24}$$

The 32 bars therefore cover a frequency range from 0 Hz up to:

$$f_{max} = 31 \times 27 \times 23.4 \text{ Hz} \approx 19.6 \text{ kHz} \tag{25}$$

corresponding to the entire audible spectrum of the human ear (20 Hz − 20 kHz).

7.3 Phase 2 — Peak Hold Update

Upon arrival of the `frame_ready` signal, generated by the `Pulse_CDC` module upon completion of each FFT frame, the circuit updates the peak register of the bar currently being scanned, namely the one corresponding to the value of `pixel_x` at the instant `frame_ready` goes to $'1'$. Since `frame_ready` lasts for a single clock cycle (40 ns) and `pixel_x` takes a random value at that instant, in each FFT frame only one bar's peak is updated. Over subsequent frames, the random distribution of update instants ensures that all 32 bars are updated within a few consecutive FFT frames.

For each bar the behavior follows three phases:

- **Attack:** If the current bar value exceeds the stored peak, the `peak_val` register is immediately updated to the new value and the decay counter `peak_decay_cnt` is reloaded to the maximum value PEAK_DECAY = 20 frames.
- **Hold:** If the bar drops but the counter is still greater than zero, the peak remains frozen at the maximum reached level and the counter is decremented by one unit per frame.
- **Release:** When the counter reaches zero, the peak drops by 1 pixel per frame, producing the controlled descent effect visible on the screen.

7.4 Phase 3 — Color Generation for Each Pixel

For every clock cycle at 25 MHz, the module evaluates the position of the current pixel with respect to the bar geometry and assigns the RGB color according to a three-level priority hierarchy.

Level 1 — Active Zone Check: If the pixel lies in the separation gap between bars (`in_bar` = $'0'$) or in the blanking zone (`video_active` = $'0'$), the RGB buses are forced to zero producing the black background color.

Level 2 — Peak Indicator: If the pixel lies within the 3-pixel vertical window corresponding to the stored peak level:

$$(479 - \text{current_peak}) \leq py \leq (479 - \text{current_peak} + 2) \tag{26}$$

the pixel is colored in **orange**, producing the peak indicator line.

Level 3 — Bar Body with Color Gradient: If the pixel lies within the body of the bar, that is:

$$py \geq (479 - \text{bar_height_reg}) \tag{27}$$

the color is assigned based on the overall bar height, emulating an LED VU-Meter:

- Height > 360 pixels $\implies$ **Red** — clipping level
- Height > 240 pixels $\implies$ **Dark Orange** — high level
- Height > 120 pixels $\implies$ **Yellow** — medium level
- Height ≤ 120 pixels $\implies$ **Green** — normal level

7.5 Temporal Dynamics and Visual Perception

The period of a single pixel clock is:

$$T_{pixel} = \frac{1}{f_{VGA}} = \frac{1}{25 \times 10^6} = 40 \text{ ns} \tag{28}$$

A complete frame includes both visible pixels and horizontal and vertical blanking periods. The 800 horizontal cycles consist of 640 visible pixels plus 160 blanking cycles (front porch 16 + sync pulse 96 + back porch 48). The 525 vertical lines consist of 480 visible lines plus 45 blanking lines (front porch 10 + sync pulse 2 + back porch 33), for a total of $800 \times 525 = 420000$ pixel clock cycles:

$$T_{frame} = 800 \times 525 \times T_{pixel} = 420000 \times 40 \text{ ns} = 16.8 \text{ ms} \tag{29}$$

corresponding to a refresh frequency of:

$$f_{refresh} = \frac{1}{T_{frame}} = \frac{1}{16.8 \times 10^{-3}} \approx 60 \text{ Hz} \tag{30}$$

The VGA module redraws the screen approximately 2.5 times for each spectral update, since the FFT produces a new spectral frame every:

$$T_{FFT} = \frac{N_{FFT}}{F_s} = \frac{2048}{48000} \approx 42.7 \text{ ms} \tag{31}$$

In the intermediate frames, the BRAM still contains the values from the previous frame and the bars are redrawn identically, guaranteeing a stable and artifact-free video output.

The apparent movement of the bars and peak indicators is the result of retinal persistence: the human eye merges successive frames perceiving fluid and continuous motion, even though physically every frame is completely erased and redrawn from scratch.

8 POWER CONSUMPTION

Vivado provides in the IMPLEMENTATION sections a report on Power and with this image below is possible to analyze the power consumption on chip.

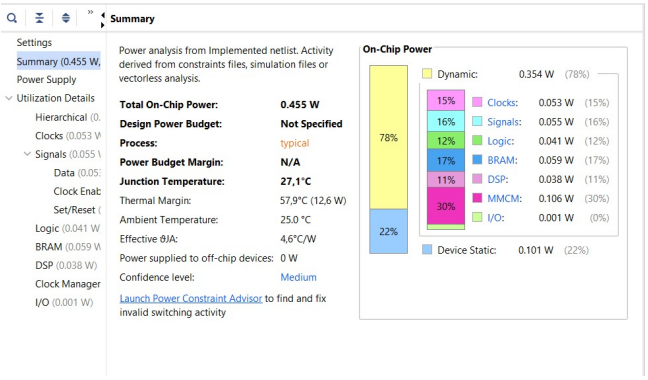


Figure 15: Power consumption on chip.

1. Global Consumption and thermal efficiency

   - The board uses less than 0.5W thanks to an optimezed datapth that combines optimeze ip core blokcs.

2. Power distribution

   - Dynamic power consumptions:

     The 78% of 0.455W is dissipated by logic commutations of transistors when the signals change. states.

   - Device static power consumption: All the board has their intrinsic power consumption due to lekeage current that flows in the transistor even if the board is not fed by any power supply.
   - The 30% of Dynamic power consumption is used by MMCM for the managment of 100Mhz clock and 25Mhz clock. In order to make vivado recognizes 25MHz as a clock it has been used a clocking wizard that generate thanks to his setting 25MHz clock fro the VGA signal and data transfer between FFT and the DISPLAY.

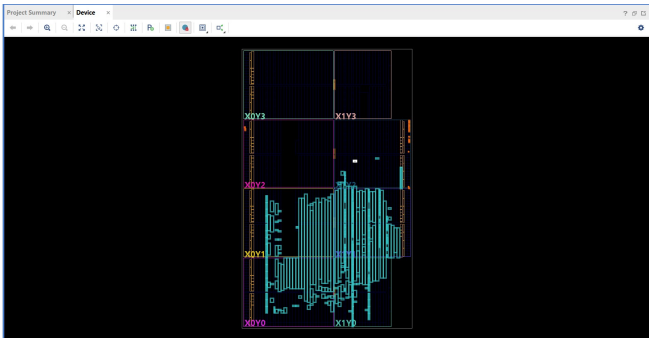


Figure 16: Resources distribution on FPGA

9 Timing Report



Figure 17: Timing analysis.

This 17 shows that time constrain are met so we can run the system at 100MHz without any violation and this is thanks to an optimized routing post implementation and optimzed blocks using IP core.

9.1 Critical Paths

- WNS(Worst Negative slack)

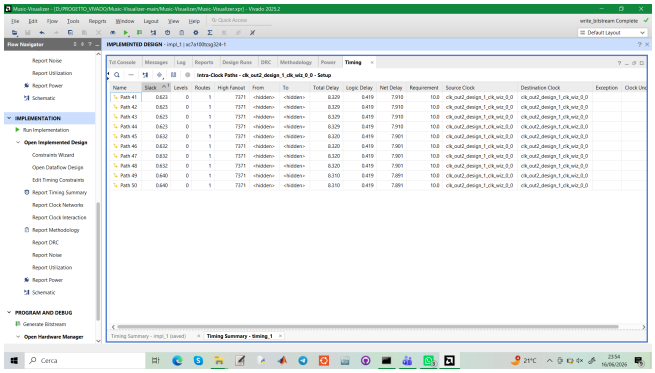


Figure 18: Paths related to WNS

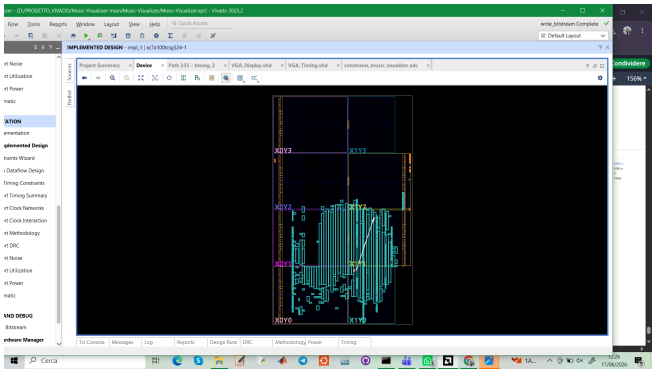


Figure 19: Critical path showed on chip paths

- WHS (Worst Hold Slack)

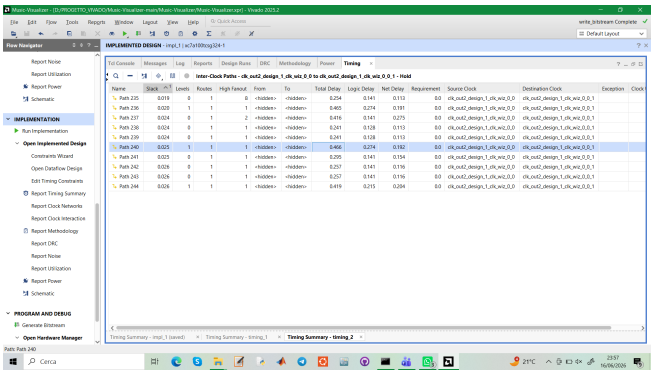


Figure 20: Paths related to WHS

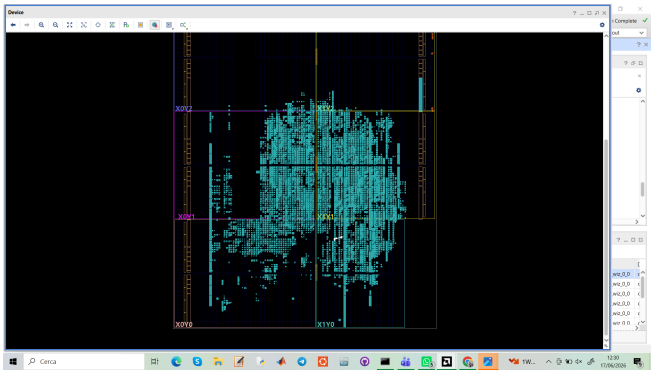


Figure 21: Critical path showed on chip paths.

Figure 18 shows the critical paths for WNS and what we see is that all the WNS are positve meaning that signals are faster than the clock. The worst path on this image is path 41 with WNS of 0.623ns.

Figure 20 shows the critical paths for WHS and what we see is that all the WHS are positve meaning that signals are stable after the rising edge of the clock. The worst path on this image is path 235 with WHS of 0.019ns.

10 ILA analysis

In order to to see if the handshake signals of the AXI Streams are working i have made some screenshot on the signals that are running on the system.

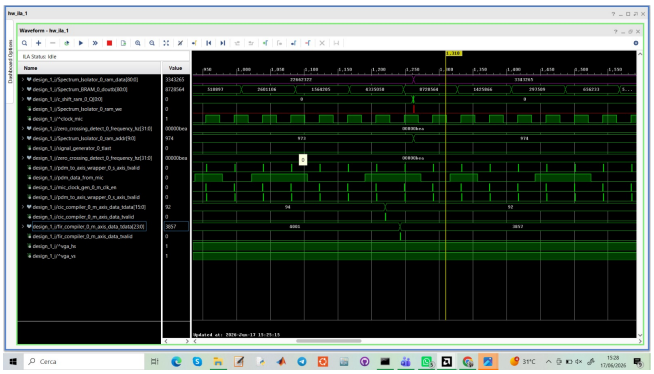


Figure 22: This screenshot shows the handshake of the Tvalid signal on CIC filter and FIR filter and signal Q that is the Tvalid of the FFT delayed of 13 clock cycle as we said before.

What we see is also the samples that exit from the Spctrum isolator and populate DUAL PORT BRAM. This the confirmation that everythings is working properly since we look only when the data is valid. Without this handshake system the system would capture false samples that wouldn't correspond to any data provided by the microphone.

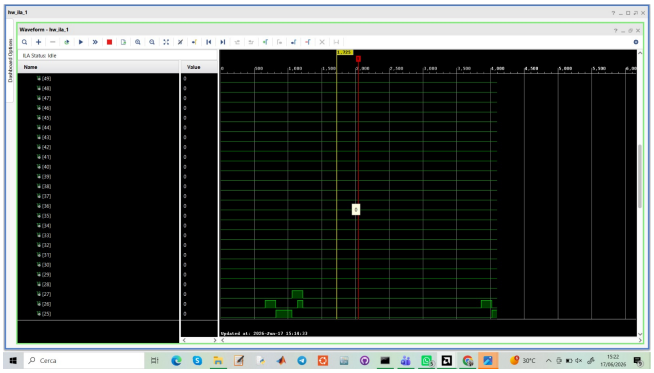


Figure 23:

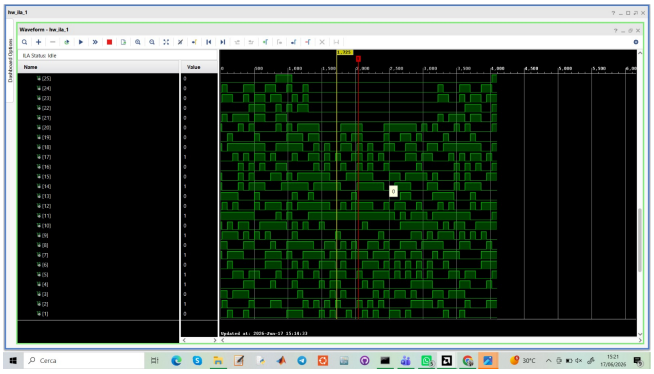


Figure 24:

Figure 25 and 24 confirms the fact that in the VGAdispaly entity the most representative bits are taken in the range between 0 and 27; in particular i have selected bits from 27 to 19. Note that if you choose a bit range where there is no information you don't see anything on the monitor.

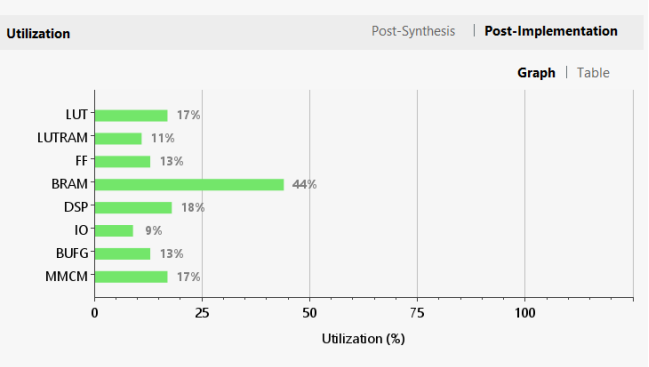


Figure 25: This images describes the resources of the FPGA in post implementation and we see that BRAM is most used due to we need to stored a large amount of data to plot correctely the bars on display

11 Conclusion

To conclude, this project successfully converges core academic methodologies from Signal Theory, Logic Networks, Physics II, and Digital Signal Transmission into a cohesive, real-time embedded system. The resulting hardware architecture translates an analog acoustic phenomenon into a highly responsive visual representation on a VGA monitor, where the dynamic deflection of each vertical bar directly corresponds to the spectral Energy (the physical volume) of the captured audio components.

Testing the system with a pure tone generator validates the theoretical design while highlighting the intrinsic limitations of real-world digital signal processing:

- **Spectral Resolution and Leakage:** Due to the finite 2048-point FFT window and a spectral resolution of $\Delta f \approx 23.4375$ Hz, a pure tone cannot be perfectly confined to a single isolated bar, an effect further pronounced by the spectral leakage of the implicit rectangular windowing in the FFT process.
- **Frequency Mapping:** Low-frequency tones activate the leftmost bars, mid-range signals near 10 KHz illuminate the center of the display, and high-frequency inputs near the audible limit of 19.6 KHz strictly drive the rightmost bars, confirming the mathematical accuracy of the BRAM addressing logic.
- **Environmental and Thermal Noise Floor:** Electromagnetic and thermal interference stemming from the monitor, cabling, and switching power supplies introduces a visible noise floor in the sub-quantization levels. While this background activity does not destroy or compromise the core audio signal, it remains visible on the highly sensitive lower-bit VGA selection window.

Ultimately, when noise falls within the processed baseband $[0, 24\text{ kHz}]$, it can't be eliminated. This stands as a textbook engineering trade-off in communication systems, demonstrating that physical constraints and noise management are fundamental pillars that must always be accounted for when bridging abstract mathematical models with physical hardware implementation.

References

[1] AMD Adaptive Computing Support. Knowledge Article 1160838. https://adaptivesupport.amd.com/s/article/1160838?language=en_US, 2024. Accessed: 2026-06-17.

[2] AMD Xilinx. Fast Fourier Transform v9.1 LogiCORE IP Product Guide (PG109) - Architecture Options. https://docs.amd.com/r/en-US/pg109-xfft/Architecture-Options, 2023. Accessed: 2026-06-17.

[3] AMD Xilinx. AMD Technical Hub Document Gateway. https://docs.amd.com/api/khub/documents/bRW3BWXLtjx3WqcTCq6k5w/content, 2024. Accessed: 2026-06-17.

[4] Dentaku1992. Audio Spectrum Analyzer In VHDL. https://github.com/Dentaku1992/AudioSpectrumAnalyzerInVHDL, 2021. Repository GitHub.

[5] Element14 Community. P2P3 AMD Vivado Cascaded Integrator Comb (CIC) Compiler - PDM Microphone to PCM Decimation. https://docs.amd.com/r/en-US/pg109-xfft/Architecture-Options, 2023. Accessed: 2026-06-17.

[6] FPGAPS. FFT IP Core Tutorial Part 1: Vivado Simulation with Complex Numbers. https://www.youtube.com/watch?v=ZdCnJutIMp8, 2025. Accessed: 2026-06-17.

[7] Tye Gardner. BRAM vivado tutorial ECE3610. https://www.youtube.com/watch?v=HPRrzgVfXo8, 2021. Accessed: 2026-06-17.

[8] MathWorks. bandpass: Bandpass filter signals. https://it.mathworks.com/help/signal/ref/bandpass.html, 2026. Accessed: 2026-06-17.

[9] MathWorks. Filter Analyzer App. https://it.mathworks.com/help/signal/ref/filteranalyzer.html, 2026. Accessed: 2026-06-17.

[10] MathWorks. Practical Introduction to Digital Filtering. https://it.mathworks.com/help/signal/ug/practical-introduction-to-digital-filtering.html, 2026. Accessed: 2026-06-17.

[11] MathWorks. Speaker Crossover Filters. https://it.mathworks.com/help/signal/ug/speaker-crossover-filters.html, 2026. Accessed: 2026-06-17.